Assignment # 01



COURSE CODE: SE-206L

COURSE TITLE:

Software Construction & Development

ASSIGNMENT ASSESSMENT TABLE:

Obtained Marks	Total Marks

Submitted To: Engr. Rizwan Shah.

Submitted By: Muhammad Danial.

Registration – No: 24ABSWE0021.

Semester: 5th

Date: 26-09-2026.

Dept. of Software Engineering

UET Peshawar, Abbottabad Campus.

Instructor Sign : _____

Assignment # 01

Repository Link: https://github.com/danial-uet/Software-Construction-Labs./tree/master/SC%2C%20Assignment%20%23%2001

Objective

To demonstrate a practical understanding of core Object-Oriented Programming (OOP) concepts: Encapsulation, Inheritance, Polymorphism, and Abstraction by refactoring vulnerable code, building dynamic behavior models, enforcing interface contracts, and critically reviewing AI-generated software designs.

Task 1 — The 'Broken Vault' (Encapsulation)

Objective:

Understand data hiding and why public fields are dangerous in software construction.

The Problem

Below is a poorly designed Java class representing a user's digital wallet:

```
public class DigitalWallet {
    public String accountHolder;
    public double balance;
    public String pinCode;
}
```

Your Task:

1. Refactor: Rewrite the DigitalWallet class to strictly enforce Encapsulation.

2. Business Rules:

• The balance can never be negative.

• The pinCode can only be set once during object creation (via a constructor) and cannot be changed or viewed directly later.

• Provide a method boolean withdraw(double amount, String enteredPin) that only processes the withdrawal if the PIN matches and sufficient funds exist.

Solution

For 1st task, 02 Java Classes were created by me inside task1 Package in SC-Assignment-1-OOP Project:

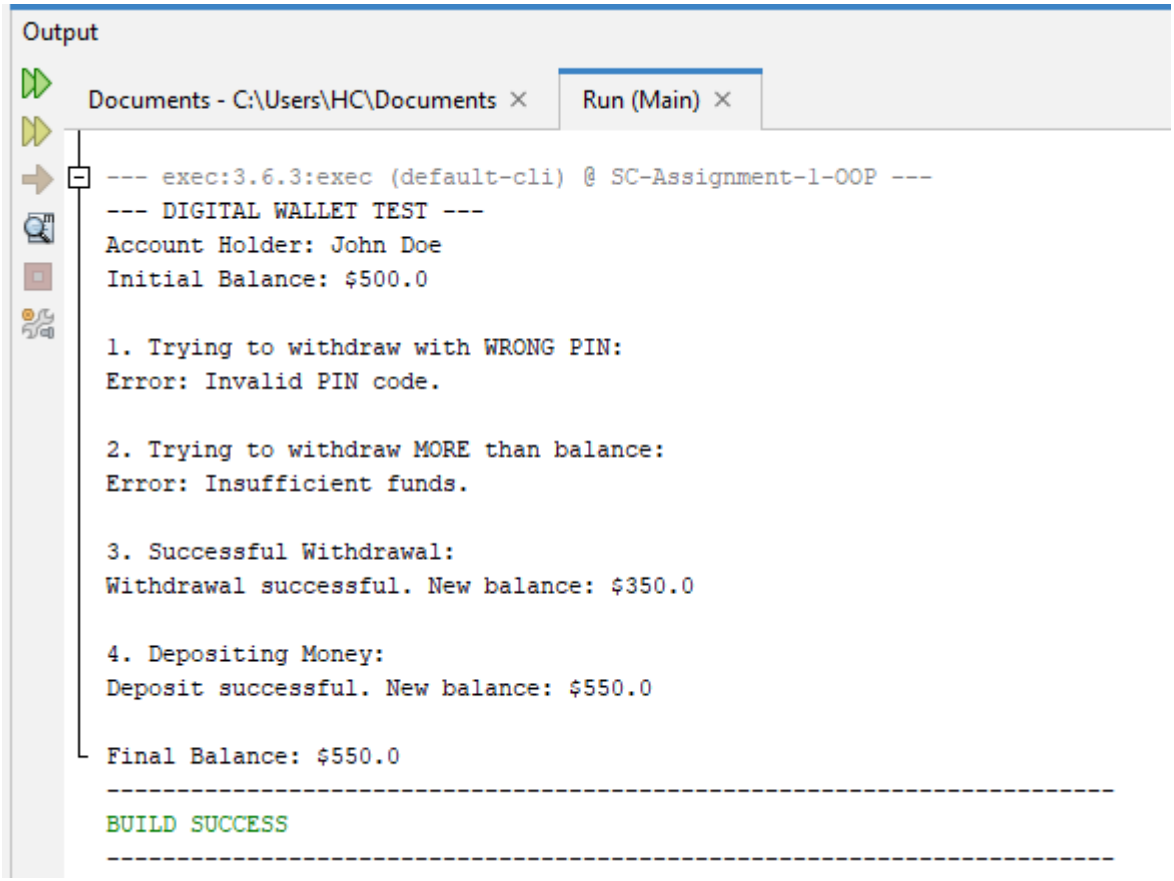
● DigitalWallet.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task1/DigitalWallet.java

● Main.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task1/Main.java

Output:



3. Reflection (Write 3-4 sentences): Explain a specific scenario where the original, unencapsulated code could cause a catastrophic failure in a real banking application.

Answer:

In the original unencapsulated code, public access modifiers allowed direct external modification of balance and exposed pinCode without validation. In a real banking application, an attacker or software bug could directly overwrite balance to arbitrary amounts or cause negative balances, bypassing fraud detection checks. Furthermore, exposing pinCode publicly allows unauthorized classes to read sensitive credentials directly from memory. Enforcing encapsulation by making fields private and restricting state changes through withdraw() prevents these unauthorized mutations.

Task 2 — The 'Evolving Workforce' (Inheritance & Polymorphism)

Objective:

Demonstrate code reusability (Inheritance) and dynamic behavior (Polymorphism).

Your Task:

1. Base Class: Create an Employee class with fields name, baseSalary, and a method calculatePay().

2. Derived Classes:
• Create a Developer class that extends Employee. Their pay includes a fixed techAllowance.

• Create a SalesManager class that extends Employee. Their pay includes a commission **based on sales.**

3. The Polymorphism Test: Write a Main class. Create an array or List of type Employee that contains both Developers and SalesManagers. Loop through the list and print the final pay for each.

Solution

For 2nd task, 04 Java Classes were created by me inside task2 Package in SC-Assignment-1-OOP Project:

● Employee.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task2/Employee.java

● Developer.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task2/Developer.java

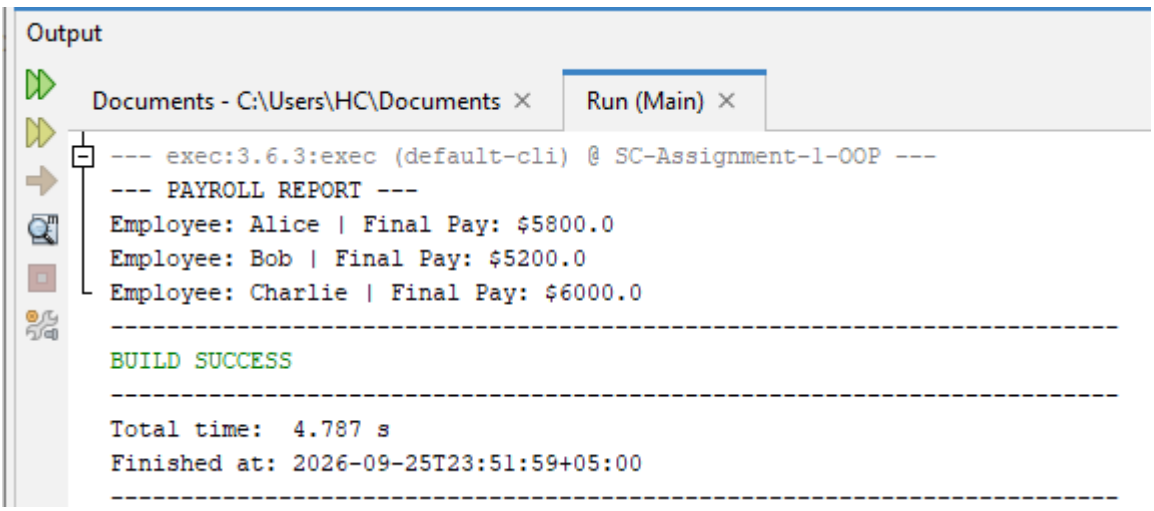
● SalesManager.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task2/SalesManager.java

● Main.java, (Junit test class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task2/Main.java

Output:



4. Reflection (Crucial Step):

When iterating through the generic List<Employee> containing Developer and SalesManager objects, Java uses Dynamic Method Dispatch to determine which pay formula to execute at runtime. Even though the reference variable emp is declared as type Employee, the Java Virtual Machine (JVM) checks the actual object instantiated in memory (created via new Developer(...) or new SalesManager(...)). When emp.calculatePay() is invoked, the JVM dynamically routes the call to Developer's calculatePay() to add techAllowance, or to SalesManager's calculatePay() to add commission, ensuring correct behavioral execution without manual type checks.

Task 3 — 'Design by Contract' (Abstraction)

Objective:

Understand how Interfaces force classes to guarantee specific behaviors.

Your Task:

1. The Contract: Create a Java interface named SmartDevice with three methods: turnOn(), turnOff(), and getStatus().

2. The Implementations:

• Create a SmartBulb class that implements SmartDevice. It should have an additional unique method setBrightness(int level).

• Create a SmartThermostat class that implements SmartDevice. It should have an additional unique method setTemperature(double temp).

Solution

For 3rd task, 04 Java Classes were created by me inside task3 Package in SC-Assignment-1-OOP Project:

● SmartDevice.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task3/SmartDevice.java

● SmartBulb.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task3/SmartBulb.java

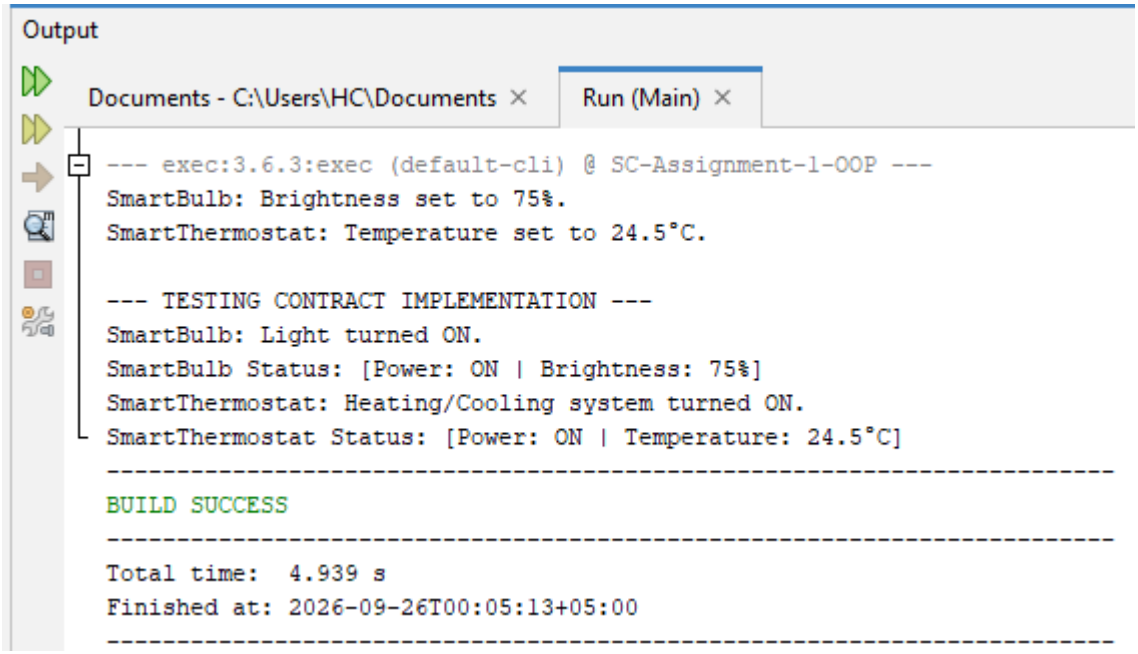
● SmartThermostat.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task3/SmartThermostat.java

● Main.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task3/Main.java

Output:



3. Reflection: Why did we use an interface here instead of just creating a standard SmartDevice class and inheriting from it? What specific advantage does this give the software architect?

Answer:

An interface was used instead of a standard base class because SmartBulb and SmartThermostat share common operational behaviors (turnOn, turnOff, getStatus) without sharing state or execution logic.
In Java, classes are restricted to single class inheritance; using a base class would force hardware devices into a rigid inheritance tree and prevent them from extending other classes.
For a software architect, using an interface enforces a strict "contract".
It guarantees that any future device (e.g., SmartLock, SmartCamera) can be seamlessly plugged into central automation systems (like smartHomeHub) as long as it fulfills the SmartDevice contract, promoting loose coupling and scalability without altering central application logic.

Task 4 — The AI Code Review (Meta-Learning)

Objective:

Learn to critically evaluate and correct AI-generated code using OOP principles.

Your Tasks:

1. Prompt an AI: Go to ChatGPT, Gemini, or Claude. Give it the following prompt exactly: "Write a Java program for a simple Library System using OOP. Include classes for Book and Member."

2. Analyze the Output:
- Identify one thing the AI did well regarding OOP principles (e.g., good use of constructors, proper encapsulation).
- Identify one design flaw, missing OOP principle, or bad practice in the AI's code.

3. The Fix: Write the corrected version of the specific class or method you identified as flawed.

Solution

For 4th lab task, 02 Java Classes were created by me inside task1 Package in SC-Assignment-1-OOP Project:

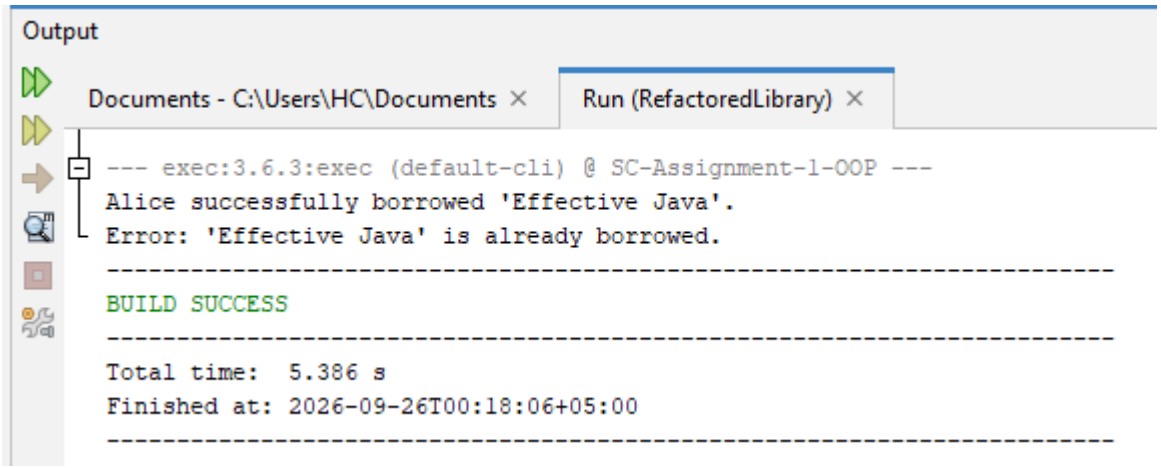
● FlawedLibrarySystem.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task4/FlawedLibrarySystem.java

● RefactoredLibrary.java, (Java Class):

https://github.com/danial-uet/Software-Construction-Labs./blob/master/SC%2C%20Assignment%20%23%2001/src/main/java/task4/RefactoredLibrary.java

Output:



Task 4 Reflection (AI Code Review):

• The AI properly identified domain entities (Book and Member) and created cohesive classes with constructors to initialize mandatory fields.

• However, it violated encapsulation by marking fields like isBorrowed and borrowedBooks as public, allowing unvalidated external state modification.

• It also lacked business logic checks, directly setting isBorrowed = true in borrowBook() without checking if the book was already checked out.

• Refactoring the fields to private and wrapping operations in validated methods prevents state corruption and enforces domain rules.

CONCLUSION:

This assignment demonstrated how core OOP principles: Encapsulation, Inheritance, Polymorphism, and Abstraction; ensure code security, reusability, and modularity. It also highlighted that while AI tools can assist with basic syntax, critical developer analysis is essential to enforce robust design patterns and business logic.
